

~ Codex Hash Research Laboratory · Whitepaper Series · 2026

ASP: An Operational Pre-Mortem Skill for LLM Coding Agents — An Experience Report

Author · Carlos Ulisses Flores CTO & Chief Researcher · Codex Hash Research Laboratory · São Paulo, Brazil MSc Program in Artificial Intelligence (in progress) · American Global Tech University ORCID: https://orcid.org/0000-0002-6034-7765 · Lattes: http://lattes.cnpq.br/6905246706890561 c.ulisses@gmail.com · https://ulissesflores.com

Whitepaper · **Version 2.0** · **2026-11-07** *(first published 2026-05-18; revised 2026-07-09)* **DOI** https://doi.org/10.5281/zenodo.20276631 *(concept · always resolves to latest version)* **Status** Independent technical whitepaper · not under peer review **License** Paper text, figures and data — CC BY 4.0 · Companion software — MIT **Replication** Every number in Sections 5.3–5.4 is re-derived by `python run_all.py` in the companion repository and asserted by a test; see Appendix B. **Companion repository** https://github.com/ulissesflores/anticipating-shadow-points **Companion whitepaper** Flores, C. U. (2026b). *Graceful Refusals as Unsignalled Failures: Characterising `claude -p` Exit-Code Semantics on Agent Declines.* See `paper/iron-law-11.md`.

> **Correction note (version 2.0, 2026-11).** Version 1.0 (Zenodo v1.0.0–v1.0.3) reported in Section 5.2 a deterministic match of 4% (RED) versus 80% (GREEN) as the adjudicator's reading of "ten candidate transcripts" produced by Claude Opus 4.7. That was wrong. The published benchmark run (`benchmark/runs/published/2026-05-18T00-27-43Z-b0a1bc7b`) is a **mock-mode** run: its manifest records `"mode": "mock"`, and every candidate text is a fixture the author wrote to exercise the adjudicator's matching layers, as the benchmark's own protocol states (`benchmark/PROTOCOL.md`, section 8: the fixtures "are NOT representative of true LLM outputs"). The 4% / 80% figures and the label-permutation control (80% to 0.5%) are therefore checks of the instrument on constructed text, not evidence about ASP. The only evidence about ASP's effect is the author's non-blind reading of the 2026-05-17 subagent dispatches, whose raw transcripts were not preserved. Section 5 is rewritten accordingly; the version 2.0 replication layer labels the data itself (`adjudicator_fixture_check_2026_05`, `run_mode: mock`).

Abstract

Software teams increasingly delegate routine engineering work to LLM coding assistants. When the assistant produces *locally correct* code that omits failure modes a senior engineer would surface in a pre-mortem — the row-level security policy that needs an update for a new column, the rate limit on the external API, the replica lag during a long migration — production breaks in ways that are expensive to debug. We call these missing items *shadow points*. This experience

report describes **Anticipating Shadow Points (ASP)**, a 13-phase planning protocol shipped as a Claude Code skill that integrates Klein's (2007) pre-mortem with the multi-agent failure taxonomy of Cemri et al. (2025), separates planning from validation under prompt isolation grounded in the self-correction-limit literature (Huang et al., 2024; Tyen et al., 2024), and hands execution to a child subprocess with a file-based contract. We document the design, the operational deployment as an open-source plugin, and preliminary observations from five engineering evaluations (single model; GREEN dispatched on all five tasks, RED on three; read by the author without blinding). The raw transcripts of those dispatches were not preserved, so the observations are reported, not verifiable. A deterministic adjudicator with cryptographic provenance ships with the skill. This version corrects an error of version 1.0, which presented the adjudicator's scores on author-constructed test fixtures as a reading of the model transcripts; those scores validate the instrument and nothing more. We therefore report **no measured effect** of ASP. A real-mode benchmark with independently authored, blind expected lists, drafted for pre-registration, is specified as the decisive follow-up. A separate empirical finding on `claude -p` exit-code semantics under graceful agent refusal is characterised in a companion paper.

Keywords: llm coding agents; pre-mortem; multi-agent failure modes; claude code; operational protocols; experience report

Resumo

Equipes de software delegam cada vez mais trabalho de engenharia rotineiro a assistentes de código baseados em LLM. Quando o assistente produz código *localmente correto* que omite modos de falha que um engenheiro sênior levantaria num pre-mortem — a política de segurança por linha que precisa mudar por causa de uma coluna nova, o limite de taxa da API externa, o atraso de réplica durante uma migração longa —, a produção quebra de formas caras de depurar. Chamamos esses itens ausentes de *shadow points*. Este relato de experiência descreve o **Anticipating Shadow Points (ASP)**, um protocolo de planejamento em 13 fases distribuído como skill do Claude Code, que integra o pre-mortem de Klein (2007) à taxonomia de falhas multiagente de Cemri et al. (2025), separa planejamento de validação sob isolamento de prompt, apoiado na literatura sobre os limites da autocorreção (Huang et al., 2024; Tyen et al., 2024), e entrega a execução a um subprocesso filho com contrato em arquivo. Documentamos o desenho, a implantação como plugin de código aberto e observações preliminares de cinco avaliações de engenharia (um modelo; GREEN nas cinco tarefas e RED em três, lidas pelo autor sem cegamento). As transcrições brutas dessas execuções não foram preservadas, então as observações são relatadas, não verificáveis. Um adjudicador determinístico com proveniência criptográfica acompanha a skill. Esta versão corrige um erro da versão 1.0, que apresentou as notas do adjudicador sobre fixtures de teste escritos pelo autor como se fossem a leitura das transcrições do modelo; essas notas validam o instrumento e nada mais. Por isso **não relatamos efeito medido** do ASP. Um benchmark em modo real, redigido para pré-registro, com listas esperadas escritas por revisores independentes e cegos, fica especificado como o próximo passo decisivo. Um achado empírico separado, sobre a semântica do código de saída do `claude -p` quando o agente recusa com educação, é caracterizado num artigo complementar.

Palavras-chave: agentes de código llm; pre-mortem; modos de falha multiagente; claude code; protocolos operacionais; relato de experiência

---

1. Introduction

Software teams increasingly delegate routine engineering work — adding a column to a production database, deploying a function that calls an external API, refactoring a utility used across thirty files — to AI coding assistants. When the assistant gets it right, the engineer's day is freed. When it gets it *subtly* wrong — by writing code that compiles, passes its tests, ships, but quietly omits a step a senior engineer would have flagged in advance — production breaks in ways that are expensive to debug.

The dominant production failure mode of modern LLM coding agents, in our experience, is not *bad* code. It is *incomplete* code: the agent does what was asked, but not what someone with deep domain context would have asked themselves *before starting*. The replica lag during a schema migration. The CORS header on the edge function. The row-level security policy that needs updating to match the new column. We call these missing items **shadow points** — failure modes obvious to a practitioner who has been burnt by them, invisible to an agent that has not.

Two streams of literature converge on this gap. The first, in decision science, is the **pre-mortem**: a structured exercise in which a team imagines a project has already failed and enumerates causes before acting. The technique was operationalized for project teams by Klein (2007) and grounded in the earlier empirical work of Mitchell, Russo and Pennington (1989) on prospective hindsight; Klein (2007) summarises it as raising the ability to identify reasons for a future outcome by about 30%. The second, in agent research, is the recent characterization of multi-agent failure modes by Cemri et al. (2025), who derived a 14-mode taxonomy from 150 seed traces of multi-agent LLM systems (inter-annotator agreement κ=0.88) and released a corpus of 1,600+ annotated traces. Neither stream has been integrated as a **runtime discipline** that a coding agent can be made to follow on every non-trivial task.

This paper reports on **Anticipating Shadow Points (ASP)**, a 13-phase planning protocol shipped as a Claude Code skill that combines pre-mortem reasoning with the MAST 14-mode checklist, separates planning from validation under strict prompt isolation, and hands execution to a child subprocess with a file-based contract. We deployed the skill, distributed it through three install paths (plugin marketplace, dev mode, standalone script), ran it through five engineering evaluations, and report what we learned. The evaluation is preliminary and, as corrected in this version, unmeasured: GREEN on five tasks and RED on three, one dispatch each, a single model, read by the author without blinding, with the raw transcripts not preserved. We report it as qualitative observation about which design choices appear load-bearing, not as an effect.

Contributions

This experience report contributes:

1. **The ASP protocol as documented design** — thirteen phases, each emitting a named artifact, organised around the principle that no implementation begins before shadow points have been surfaced.
2. **Twelve operational constraints we call "Iron Laws"** that render the protocol enforceable at runtime, each tied to a primary literature source or to a documented design decision.
3. **Preliminary, qualitative observations** from five engineering evaluations, and a deterministic adjudicator with cryptographic provenance, validated on constructed fixtures and ready to score real transcripts without trusting the author's adjudication (the expected lists and synonym sidecar are still the author's).

4. **A separate empirical finding** on `claude -p` exit-code semantics under graceful agent refusal, characterised in the companion paper Flores (2026b). The finding motivated two of the Iron Laws but warrants independent characterization at larger N.

This is an **experience report** in the systems sense, not a benchmark paper. We document a system we built, a methodology we deployed, and what we observed; we do not claim a robust empirical contribution to the LLM-agents benchmark literature. Section 7 enumerates the threats to validity in detail.

---

2. Background

We review four threads of recent prior work that directly inform ASP's design (§§2.1–2.4), then situate ASP within the older software- and quality-engineering disciplines it transposes (§2.5). We keep the recent-work review short relative to typical agent-systems papers, because the contribution here is operational and the literature context is in service of that.

2.1 Pre-mortem and prospective hindsight

The pre-mortem technique asks a team to imagine, before commitment, that a project has already failed catastrophically, and to enumerate causes. The empirical basis is Mitchell, Russo and Pennington (1989), who showed that imagining a future event as certain (rather than uncertain) yields explanations that are longer and more episodic; Klein (2007) summarises the effect as about a 30% gain in correctly identifying reasons for a future outcome. Klein (2007) operationalized the finding as the project-team "pre-mortem" and the technique has been standard in decision-science and project-management practice since. To our knowledge, it has not previously been integrated as runtime discipline in agentic LLM frameworks.

2.2 Multi-agent failure modes (MAST)

LLM agents interleave reasoning and acting in a single loop (Yao et al., 2023), and are increasingly composed into multi-agent workflows whose failure modes compound. Cemri et al. (2025) introduce the **Multi-Agent System Failure Taxonomy (MAST)** with **MAST-Data**, a corpus of 1,600+ annotated traces from seven popular multi-agent frameworks such as AG2 (formerly AutoGen; Wu et al., 2023). From rigorous analysis of 150 seed traces (inter-annotator agreement κ=0.88), they derived fourteen failure modes organised into three categories: specification and system design (5 modes), inter-agent misalignment (6 modes), and task verification and termination (3 modes). The 14-mode list is the strongest recent empirical grounding for a failure-mode checklist; ASP adopts the mode names verbatim and operationalizes each into a pre-mortem prompt plus mitigation in its companion skill files. Two concurrent works extend the taxonomy direction: Zhu et al. (2025) introduce AgentDebug and the AgentErrorTaxonomy (memory / reflection / planning / action / system-level), and Zhang et al. (2025), an ICML 2025 Spotlight, opens **automated failure attribution** with the Who&When dataset.

2.3 Self-correction limits and validator isolation

The strongest single argument for separating the validator from the planner is the convergent finding across three primary works that LLMs cannot reliably critique themselves under shared context. Huang et al. (2024, ICLR) show that intrinsic self-correction without external feedback

often degrades performance. Tyen et al. (2024) decompose self-correction into localization and correction and show empirically that LLMs cannot *localize* their own reasoning errors but can correct them given the location externally — *localization is the bottleneck*. Zheng et al. (2023, NeurIPS D&B) document that strong-LLM judges over-favour same-family-model outputs (a "self-enhancement bias") even when objectively judging quality. Kamoi et al. (2024) survey this literature and conclude that reliable self-correction of reasoning errors requires external feedback or verifiers rather than the model's unaided judgment; intrinsic self-feedback methods such as Self-Refine (Madaan et al., 2023) improve some generation tasks but do not close the localization gap that Tyen et al. (2024) isolate. Together these findings argue that a validator sharing context with a planner inherits the planner's confidence calibration on shared content, including erroneous content the planner is confident about. ASP's design enforces strict prompt isolation between planner and validator (Iron Law 2 below) as the architectural counter-move.

2.4 Closest related work: PreFlect

The closest concurrent work is **PreFlect** (Wang et al., 2026), which similarly shifts the self-reflection paradigm from post-hoc correction to pre-execution criticism of agent plans, distilling planning errors from historical agent trajectories and adding a dynamic re-planning mechanism. The high-level similarity is significant: both ASP and PreFlect identify the post-execution-only nature of conventional reflection as a limitation and shift criticism *before* the action. ASP differs from PreFlect along four axes:

1. **Prompt-isolated validator.** PreFlect distils errors into the agent's own pre-execution criticism; ASP enforces a separate-prompt validator subagent that never sees the planner's transcript.
2. **Process-level isolation for the executor.** ASP spawns the executor as a `claude -p` subprocess with a file-based contract; PreFlect operates within a single process.
3. **Hard runtime constraints.** ASP carries twelve non-negotiable runtime invariants enforced as Iron Laws; PreFlect is presented as a methodology rather than a constraint system.
4. **An empirical CLI-level finding** characterising `claude -p` exit semantics under graceful refusal, motivating two of the Iron Laws; this is reported separately in Flores (2026b). The phenomenon itself is documented by the vendor and in public issue reports; the residual contribution there is a measured count, not a discovery.

2.5 Classical software and quality-engineering lineage

The four threads above place ASP within the recent LLM-agents literature. A reviewer from software or quality engineering will, however, recognise ASP's machinery as a transposition of disciplines that predate LLMs by decades. We make the borrowing explicit, because concealing it would be both dishonest and weaker: what is new in ASP is not the disciplines but their transposition to a non-deterministic agent runtime and the mechanisms that enforce them there.

Design by Contract — and where ASP departs from it. ASP's micro-TODO schema (§3.4) is Design by Contract (Meyer, 1992) relocated from the routine boundary to the agent-task boundary. Meyer's contract assigns obligations across a caller/routine boundary: the caller must establish the PRECONDITION, which the routine is then entitled to *assume* — Meyer's non-redundancy principle forbids the body from re-checking it — and the routine must guarantee the POSTCONDITION on which the caller relies. Of the schema's seven fields, PRECONDITION, ACTION and POSTCONDITION map onto this contract directly, with ACTION as the routine

body the pre/post pair wraps. ASP then *departs* from Meyer on purpose: because the obligor is a non-deterministic model rather than a compiled routine, ASP does not assume the postcondition — it re-verifies it with an external ACCEPTANCE-TEST, and adds a FALSIFICATION-TEST that has no contract-programming analog, importing instead the Popperian stance (Popper, 1959/2002) that a claim earns confidence by surviving a disconfirmation attempt rather than by a confirming check alone. This trade of Design by Contract's non-redundancy for defensive re-verification is the same trust-nothing ethos that produces the exit-code discipline of Iron Laws 11–12; it is a considered inversion of Meyer, not an oversight. ASP's always-hold Iron Laws, in turn, play the role of DbC's third contract element — the class invariant — rather than a per-task pre/post condition.

Failure-mode enumeration. Shadow-point detection (Phase 2) is a prospective failure-mode enumeration in the tradition of FMEA (MIL-STD-1629A, 1980; IEC 60812:2018). It is deliberately *not* full FMEA: ASP performs no per-component decomposition and no risk-priority scoring (severity × occurrence × detection), so the debt is to FMEA's prospective-enumeration discipline, not its bottom-up mechanics. Three inputs to Phase 2 occupy roles worth separating: the pre-mortem (Klein, 2007; §2.1) supplies the *philosophy* — imagine the failure as already certain; the MAST fourteen-mode taxonomy (Cemri et al., 2025; §2.2) supplies the *library* of failure modes; and the forced enumeration against that fixed library is closer, structurally, to HAZOP-style guide-word prompting than to component-by-component FMEA. What ASP adds is the substrate — the "component" whose failure modes are enumerated is an LLM plan, elicited from the same model under an isolation constraint.

Mistake-proofing. A subset of the Iron Laws (§3.3) are poka-yoke (Shingo, 1986) at the protocol level: Iron Law 9's `ASP_IN_GOAL` env-guard blocks reentrant recursion by default, and Iron Laws 10 and 12 make a file-based contract and JSON output mandatory rather than optional. We claim no more than a subset. These are runtime guards, not the physical impossibility of true poka-yoke — an environment flag can be unset or fail to propagate — and the remaining laws are honestly procedural gates (Iron Laws 1, 2, 4, 5) or verification rules that still depend on the operator, of which Iron Law 11 ("never trust `$?`") is the clearest case: precisely the operator-vigilance dependence poka-yoke is defined against. The borrowing is the mistake-proofing *stance* applied to a subset of agent-runtime invariants, not a claim that ASP eliminates operator vigilance.

Independent verification and validation. ASP's prompt-isolated validator (Iron Law 2) transposes the defining move of Independent Verification and Validation (IV&V; IEEE Std 1012-2016) — a verifier kept independent of the developer, in IEEE 1012's terms technically, managerially and financially — from organisational separation to *prompt* separation: the validator subagent shares no context with the planner it checks. The LLM-specific justification for that isolation is the self-correction literature of §2.3; the *form* of the counter-move is IV&V's.

Dependability vocabulary. The exit-code behaviour behind Iron Laws 11–12 has an exact name in the dependability taxonomy: an exit 0 on a graceful decline is an *unsignalled failure* (Avizienis et al., 2004), neither a crash nor a fail-silent stop. The term, its prior art, and the measurement are developed in the companion paper (Flores, 2026b); this section imports only the vocabulary and does not re-open the novelty question here.

Empirical warrant. The preference for acceptance and falsification tests over trusted status codes has an empirical basis in the adjacent pathology. Yuan et al. (2014) found that a large majority of catastrophic failures in distributed data-intensive systems trace to faulty handling of non-fatal errors that *were* signalled. ASP's exit-code case is the complement — the signal was *absent* — so

the two do not share a thesis; the narrow lesson they do share is that a status channel is not a proxy for outcome, which is what an external ACCEPTANCE-TEST exists to enforce.

Table 1 maps each ASP mechanism to its classical ancestor and states what ASP adds.

Table 1. ASP mechanisms and their classical ancestors.

ASP mechanism	Classical ancestor	What ASP adds (and does *not* claim)
micro-TODO PRE/ACTION/POST + ACCEPTANCE-TEST	Design by Contract (Meyer, 1992)	a non-deterministic obligor whose postcondition is re-verified by external command, not assumed; not the contract idea
micro-TODO FALSIFICATION-TEST	Popperian falsification (Popper, 1959/2002)	a disconfirmation step at the agent-task boundary; no Design-by-Contract analog
Shadow-point detection (Phase 2)	FMEA-style enumeration (MIL-STD-1629A; IEC 60812:2018) + HAZOP guidewords, over the MAST library (Cemri et al., 2025)	enumeration of an LLM plan under isolation; no risk-priority scoring, no per-component decomposition
Iron Laws 9 / 10 / 12 as structural guards	Poka-yoke (Shingo, 1986)	mistake-proofing of a subset of agent-runtime state; runtime guards, not physical impossibility
Prompt-isolated validator (Iron Law 2)	IV&V (IEEE Std 1012-2016)	organisational independence recast as prompt isolation; not IV&V
Never-trust-`$?` (Iron Laws 11–12)	Unsignalled failure (Avizienis et al., 2004); Yuan et al. (2014)	characterised in the companion paper (Flores, 2026b); not the failure class

Read together, the section makes a deliberately modest claim: ASP's machinery is an *integration* of established reliability and quality-engineering disciplines — contract, failure-mode enumeration, IV&V, mistake-proofing — into one protocol whose contribution is operational rather than conceptual. What that integration adds beyond its ancestors is their enforcement in a non-deterministic agent runtime: the runtime-checked Iron Laws, the prompt- and process-level isolation of §3.2, and the reproducible deterministic adjudicator of §5, with the `claude -p` exit-code finding characterised separately in the companion paper. This is the experience-report claim of §1, not a novelty claim against the classical literature.

3. The ASP Protocol

3.1 Overview

ASP is a 13-phase protocol that progresses from intake through delivery sign-off. Each phase emits a named artifact (Table 2). The principle organising the phases is that no implementation begins before shadow points have been surfaced (Phase 2), validated independently (Phase 5), and approved by the user (Phase 7); execution is delegated to a child subprocess with a file-based contract (Phase 9).

Table 2. ASP phases and their emitted artifacts. Phase 0 has two sub-phases (0a before research, 0b after it), so thirteen phases occupy fourteen rows.

#	Phase	Artifact	Design rationale
0a	Pre-research Q&A	`intake` block	Anthropic Skills best-practice: verifiable intermediate outputs
1	Parallel research	Research summary + recall block	Subagent dispatch pattern
0b	Post-research Q&A	Refined intake (skippable)	Same as 0a
2	Shadow-point detection	Categorised list + mitigations	Klein (2007) + Cemri et al. (2025)
3	Project Charter	Filled charter	Durable artifact-based handoff
4	Macro plan + register	Macro plan + deliverables D01–Dn	Plan-and-Act (Erdogan et al., 2025)
5	Independent validator	APPROVE / REVISE	Huang et al. (2024) + Tyen et al. (2024) + Zheng et al. (2023)
6	Internal loop (cap=3)	Final-revision plan	Bounded iteration; FM-1.5 / FM-3.1
7	User approval	Go / no-go	Human-in-the-loop gate
8	Micro-TODO + Goal Spec	`TaskCreate` per step + Goal Spec	Executor handoff
9	Execution via `/goal` subprocess	Tasks marked completed with evidence	Process-level isolation; see Flores (2026b)
10	Per-deliverable sign-off	Each deliverable accepted or rejected	Acceptance contract
11	Execution Report + memory	Closure artifact + memory write-back	Reflexion-style memory (Shinn et al., 2023)
12	(Opt-in) public release	Plugin marketplace + GitHub release	Distribution layer

3.2 Architectural positioning

ASP separates three agent roles with explicit artifact-only handoffs (Figure 1):

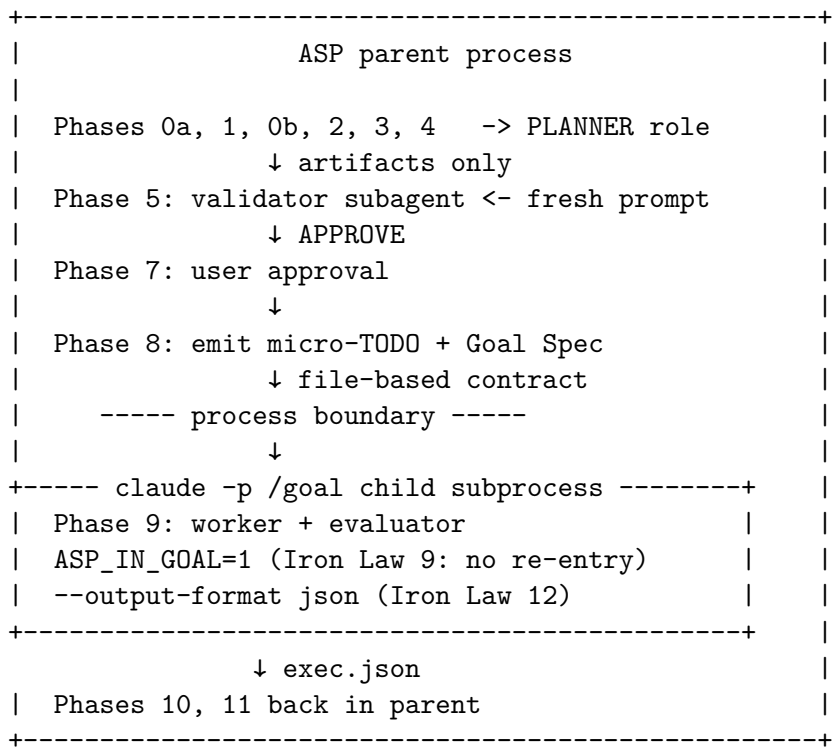


Figure 1. ASP's three-role separation. Planner phases run in the parent. The validator runs in the parent but in a fresh subagent prompt that receives only artifacts, never the planner's transcript. The worker runs in a child `claude -p` subprocess with a file-based contract.

3.3 Iron Laws

We render twelve operational constraints as non-negotiable runtime invariants and call them Iron Laws (Table 3). Each is mapped to its derivation: a primary literature source, a documented Anthropic recommendation, or an empirical design decision.

Table 3. The 12 Iron Laws and their derivation.

#	Law	Derivation
1	No implementation without shadow-point pass first	Klein (2007) + Cemri et al. (2025)
2	No plan approval without independent validator pass (separate prompt)	Huang et al. (2024) + Tyen et al. (2024) + Zheng et al. (2023); Du et al. (2024)
3	No micro-TODO without macro plan + charter + deliverables approved by user	Anthropic Skills: verifiable intermediate outputs
4	No step completion without fresh acceptance-test evidence	Cemri et al. (2025) FC3 (FM-3.2)

#	Law	Derivation
5	No deliverable accepted without evidence file/output matched	Cemri et al. (2025) FC3 (FM-3.3)
6	No project close without execution report + memory write-back	Shinn et al. (2023) memory buffer
7	No loop without cap (default cap=3 validator rounds)	Cemri et al. (2025) FM-1.5 + FM-3.1
8	No Phase 9 execution without evaluator dispatch at checkpoints (except under `--quick`)	Erdogan et al. (2025) planner/executor
9	No reentrant ASP inside `claude -p /goal` child (`ASP_IN_GOAL` env-var guard)	Cost-explosion mitigation; design decision
10	No `claude -p /goal` spawn without file-based contract	Subprocess state-divergence mitigation; design decision
11	Never trust `$?` from `claude -p` for semantic success/failure	Empirical observation, characterised in Flores (2026b)
12	All `claude -p` invocations must use `--output-format json` or `stream-json`	Corollary of Iron Law 11

3.4 The micro-TODO contract

At the boundary between planning and execution (Phase 8), ASP emits a contractual task list. Each task is created via the `TaskCreate` primitive with seven labeled fields:

- **PRECONDITION** — invariant that must hold before the step.
- **ACTION** — exact command or literal edit.
- **POSTCONDITION** — state change effected by the step.
- **ACCEPTANCE-TEST** — command and expected output that prove the postcondition.
- **FALSIFICATION-TEST** — command or observation that would prove the step failed.
- **DEPENDS-ON** — upstream task identifiers (DAG structure).
- **DELIVERABLE-ID** — which deliverable in the register this step contributes to.

This schema is what makes the task list auditable: no task is marked completed without acceptance-test evidence (Iron Law 4); no deliverable is signed off without matching evidence (Iron Law 5). The schema is shared with the Goal Spec consumed by the Phase 9 subprocess.

---

4. Operational Deployment

ASP ships as a Claude Code plugin and is distributed through three install paths, each appropriate for a different audience:

1. **Plugin marketplace** (recommended for end users):

```
claude plugin marketplace add ulissesflores/anticipating-shadow-points
claude plugin install anticipating-shadow-points@anticipating-shadow-points
```

Invocation: `/anticipating-shadow-points:asp <task>` or bare `/asp`.

2. **Developer mode** (for contributors): `claude --plugin-dir ./` loads the skill in place; useful when iterating on the skill itself.
3. **Standalone install script** (for users who do not use the plugin manager): `./scripts/install.sh` copies the skill into the user's Claude Code skills directory.

The plugin uses Anthropic's official `.claude-plugin/plugin.json` manifest format (Claude Code 2.1+; Anthropic, 2026a), and the repository doubles as a self-host marketplace via `.claude-plugin/marketplace.json` at the repository root.

Permissions. Since plugin version 1.0.4 the Phase 9 runner spawns the `claude -p` child with **no permission flag**: the child runs under the user's own Claude Code permission settings, and the skill never grants, widens or bypasses them. Earlier versions passed `--permission-mode bypassPermissions`; that was removed after the Claude plugin directory declined the listing because a plugin must not grant itself permissions. In non-interactive mode an action the user has not pre-allowed is denied rather than prompted; the parent reads the `permission_denials` field of the child's JSON result and reports any denial to the user instead of retrying with wider permission. This makes autonomous execution strictly weaker than in the May 2026 deployment, and safer: the skill can only do what the operator already allowed. The skill itself is implemented as a Markdown-based skill following the Anthropic Agent Skills best-practices document (Anthropic, 2026b), with companion files for the MAST checklist, per-mode pre-mortem prompts, and the seven artifact templates referenced in Section 3.1.

Documentation is published as a GitHub Pages site, with translations of the README into Spanish, Portuguese, Italian, and Hebrew; the latter three are AI-assisted and pending native-speaker review. The full source — skill files, plugin manifests, documentation, test artefacts, benchmark code — is MIT-licensed; the paper text, figures and data are CC BY 4.0.

---

5. Preliminary Observations

We ran ASP through five structured engineering evaluations spanning heterogeneous domains. This section separates three things that version 1.0 blurred: what the author observed in real model dispatches (5.2), what the deterministic adjudicator shows about itself on constructed fixtures (5.3), and what has not been measured at all.

5.1 The evaluation set

Table 4. The five engineering evaluations.

ID	Domain	Task (abbreviated)	Expected size	Acceptance
E1	Schema migration	NOT NULL column added to 1M-row `user_profiles`	10	$\geq 80\%$
E2	Edge deploy	Edge function calling rate-limited external API	10	$\geq 80\%$
E3	Refactor	Date-formatting utility used across 30+ files	10	$\geq 80\%$
E4	RLS policy	Row-level security policy referencing new column	10	$\geq 80\%$
E5	Scheduling	Cron job whose script conflicts with disabled service	10	$\geq 80\%$

Each evaluation specifies an INPUT (a task statement as a senior engineer might phrase it to a colleague), an EXPECTED list of shadow points that a competent practitioner would surface, and an ACCEPTANCE threshold (default 80%). The full text of each evaluation is in the companion repository.

5.2 What was observed in real dispatches (author, non-blind)

On 2026-05-17 the evaluations were dispatched to fresh subagents: GREEN (with the ASP discipline injected: pre-mortem framing, MAST 14-mode forced enumeration, mitigation requirement) on all five, and RED (asked to plan as a senior engineer would, without any framework) on three, E1–E3; E4 and E5 were added afterwards and run in GREEN only. All ran on the session model of the time (Claude Opus 4.7; the model was not recorded per dispatch), one dispatch per condition. The subagents inherited the author's global agent instructions, so the RED condition was a competent baseline, not a naive one. The author read each plan against the expected list and recorded a coverage judgement; the summaries, with the subagent identifiers, are committed in `tests/baseline-pressure-tests.md` and `tests/evals-summary.md`. The raw transcripts were not committed and are no longer available.

Under that non-blind reading, the three RED plans covered roughly 30–60% of their expected lists (mean near 47%, counting partial mentions as covered) and the GREEN plans covered all or nearly all of theirs. The committed summaries predate this correction and still conclude that ASP is "reproducibly better"; that conclusion is not supported. These figures are an author's judgement on unpreserved text, scored against lists the same author wrote, with the MAST vocabulary that the GREEN prompt injects appearing in those lists. They are reported as observation, not measurement, and no number in this paragraph is asserted by the replication layer.

5.3 What the deterministic adjudicator shows about itself

The `benchmark/` package scores a plan against an expected list with a deterministic 4-layer matcher (exact name, synonym sidecar, Jaccard sliding window, unmatched). Its one published run is a **mock-mode** run: the RED and GREEN texts are fixtures the author wrote to exercise each matching layer (`benchmark/tests/fixtures/mock_runs/`), deliberately weaker for RED and deliberately aligned with the expected list for GREEN. On those fixtures the matcher scores RED 4% and GREEN 80% on average — a statement about how the fixtures were written, not about ASP.

The useful property the mock run does establish is **discrimination**: scoring each GREEN fixture against a *different* task's expected list drops the mean from 80% to 0.5% (largest off-task cell 10%; Figure 2), so the matcher keys on task-specific vocabulary rather than generic engineering words. That shows specificity, not recall: the E2 RED fixture mentions a rate limit, retries and logging and still scores 0/10, so paraphrase that shares no vocabulary with the expected item is missed. The matcher has not yet been applied to any real transcript. The real-mode protocol, with expected lists re-authored blind by independent engineers, is specified in the repository's `PREREGISTRATION.md` and has not been run.

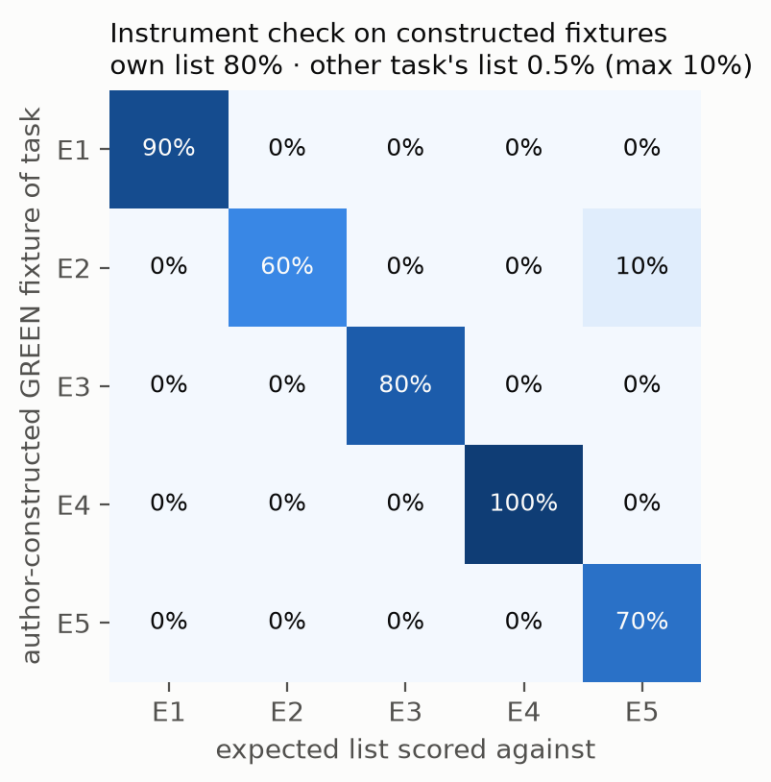


Figure 2. Instrument check: each author-constructed GREEN fixture scored against every task's expected list. Not an effect of ASP.

Which component would matter most, if ASP has an effect, is discussed as a hypothesis in Section 6.

5.4 The `claude -p` exit-code finding

During a four-test battery against the `claude -p` non-interactive runner, we observed that the binary returns exit code 0 even when the agent gracefully refuses the requested goal — the harness reports a successful conversation end despite the agent semantically refusing the task. The exit code reflects process-level success of the CLI binary, not semantic success of the agent's task. The safe-parsing recipe (require `--output-format json`, parse `is_error`, `terminal_reason`, `stop_reason`) is codified as Iron Laws 11 and 12 in the deployed skill.

This finding is observationally significant for any toolchain that wraps `claude -p` in a shell pipeline. Because the observation was from a single refusal scenario in our own battery, we **characterise it independently** in the companion paper Flores (2026b), which reports an N=50 measurement across four categories of refusal-provoking prompt (impossible-in-one-turn, capability, safety, ambiguity) on Claude Code 2.1.200 in July 2026. The session model was Fable 5; Opus 4.8 answered 16 of the 50 prompts, including all 12 safety prompts, after an automatic hand-off. Twenty-four trials exited 0, all with `stop_reason: end_turn`, and 12 to 20 of them were graceful declines: 24% to 40% of the suite depending on how a decline is counted (40%, 95% Wilson CI 28–54%, under the author's label; 38% under a blind five-rater codebook majority; with an 18% floor from an English-only phrase detector). An earlier snapshot (May 2026, Claude Code 2.1.143, Opus 4.7) read 60% under the same detector, on a different model and reply language. A human second rater and a third snapshot under a drafted pre-registration are the open follow-ups.

---

6. Lessons Learned

We highlight five design choices we believe are load-bearing. None has been measured (Section 5); each is stated as a hypothesis with the observation that motivated it.

Hypothesis: a forced taxonomy surfaces more than free-form pre-mortem. In the author's drafts, free-form pre-mortem clustered on obvious risk categories (data, integration) and underweighted less-salient ones (time/timezone, observability, operational excellence, contract drift). No dispatch tested free-form pre-mortem as a condition (RED had no framework at all), so this rests on the author's reading of drafts. The free-form prompt ("imagine the task has failed in 30 days; list 10+ causes") is necessary but not sufficient. The MAST 14-mode interrogation surfaces categories the free-form pass clusters away from.

Prompt isolation for the validator is non-negotiable. Validators that share context with planners reproduce the planner's confidence on shared content. The convergent literature on intrinsic self-correction limits, error-localization failure, and judge self-enhancement bias all point at this; we have not tested it inside ASP.

File-based handoffs survive context pressure; transcript-based handoffs do not. ASP commits charter, macro plan, deliverables register and goal spec to disk before the executor is spawned. The child receives only the Goal Spec text on stdin and returns a JSON result; the parent reconstructs state from that result and from the files on disk, so neither side depends on a transcript that may have been compressed or truncated.

Hard runtime constraints (Iron Laws) make a skill enforceable. Skills that read as "guidelines" are routinely ignored under context pressure. Skills that emit a checklist of non-negotiables — and whose templates *refuse to be filled in* without each box ticked — are, in our use, harder to

skip. Compliance has not been measured.

Treating `claude -p` exit codes as semantic-success signals is a silent footgun. This is the finding reported in Flores (2026b); mentioning it here because the design implication (Iron Laws 11 and 12) was material to the system's reliability.

---

7. Threats to Validity

We treat threats explicitly across the four categories standard in empirical software-engineering reports.

Construct validity. The same researcher who designed the protocol authored the expected-shadow-points lists *and* hand-adjudicated coverage. Even with a written rubric, this is the single largest threat to the observation's interpretability. The deterministic adjudicator we ship would mitigate this once applied to real, preserved transcripts; in this version it has only been run on constructed fixtures (§5.3), and it does not address the prior question of whether the expected lists themselves are biased toward what ASP surfaces. **Blind re-authoring by independent reviewers is the strongest follow-up.**

Internal validity. With one dispatch per cell, GREEN on five tasks and RED on three, the evaluation at most suggests that the protocol can change the transcripts on these cases; it is *not* sufficient to estimate an effect size, establish causality, or support statistical inference, and no statistical test is reported. No formal ablation of components has been run (the qualitative attribution of load-bearing-ness to the MAST checklist in §5.3 and §6 is a hypothesis). The acceptance threshold of 80% is the author's choice and not derived from prior literature.

External validity. Single underlying model (Claude Opus 4.7, presumed from the session; not recorded per dispatch), single environment (the author's macOS workstation and the Claude Code version of May 2026, not recorded per dispatch), five engineering tasks drawn from the author's domain experience. We make no claim of generalisation beyond these conditions.

Reliability. All evaluation INPUT statements, expected-shadow-point lists, the mock-run fixtures and the deterministic-adjudicator code are committed to the companion repository; the 2026-05-17 dispatch transcripts were not preserved, so the observations of §5.2 cannot be re-checked. The benchmark package builds a cryptographically-chained manifest per run so any byte edited under a run directory invalidates the manifest hash.

---

8. Limitations and Future Work

The limitations of this experience report match the threats above one-to-one. The highest-priority follow-ups, in order, are:

1. **Blind re-authoring** of the expected-shadow-point lists by 2+ independent reviewers with no exposure to ASP, then a real-mode run whose transcripts are preserved and hash-sealed, adjudicated blind. Compute Cohen's κ for inter-adjudicator agreement; re-measure coverage.
2. **Formal ablation** of the load-bearing hypothesis: MAST checklist on/off (testing the hypothesis of §6), validator prompt isolation on/off, Phase 0a intake on/off. No ablation protocol

exists yet; Protocol 2 of `PREREGISTRATION.md` covers only RED vs GREEN, at 8 replicates per cell with a fixed budget rather than a power analysis.

3. **A human second rater and a third `claude -p` snapshot** for the companion finding (Flores, 2026b, §7), the latter under Protocol 1 of `PREREGISTRATION.md`.
4. **Cross-model evaluation.** Re-run on Claude Sonnet, GPT-5, Gemini 2.5, and an open-weight model.
5. **Head-to-head against PreFlect.** Run ASP and a PreFlect reimplementation on the same task set.
6. **Embedding-based adjudicator** as a v0.2 of the benchmark, with pinned sentence-transformer model, to raise the recall floor on lexically-disjoint paraphrases.
7. **Standardized-benchmark replication.** The five-task set is not a public agent benchmark; replicating the direction on SWE-bench (Jimenez et al., 2024) or a comparable standardized suite would test whether it survives outside the author's chosen tasks.
8. **Native-speaker review** of the ES, IT, and HE documentation translations.
9. **Submission target.** Workshop venues — NeurIPS LLM-agents track, ICLR agents workshop, AAAI agents track — are the appropriate fit for the current empirical scale. Main-track submission requires the follow-ups above.

---

9. Conclusion

We described ASP, an operational pre-mortem skill for LLM coding agents, integrating Mitchell, Russo and Pennington's (1989) prospective hindsight finding, Klein's (2007) pre-mortem operationalisation, and Cemri et al.'s (2025) MAST 14-mode failure taxonomy as runtime discipline under a contract of twelve Iron Laws. The skill ships open source with three install paths. On the three tasks run in both conditions the author observed, without blinding and on transcripts that were not preserved, that the assisted plans covered more of the expected failure modes; no effect has been measured. The deterministic adjudicator shipped with the skill has been validated only on constructed fixtures, where it discriminates task-specific vocabulary; a real-mode run with independently authored lists, drafted for pre-registration, is the decisive next step.

We additionally observed that `claude -p` returns exit code 0 even when the agent gracefully refuses the requested goal — a quietly load-bearing finding for anyone wrapping the CLI in a shell pipeline. Because the observation merits independent characterisation at larger N, we characterise it in the companion paper Flores (2026b).

The strongest follow-up is blind re-adjudication. The most useful companion work is a human second rater and a pre-registered third snapshot of the `claude -p` measurement (Flores, 2026b).

Acknowledgments

This experience report was prepared in explicit human-in-the-loop collaboration with Claude (Anthropic, Opus 4.7), used as a drafting, verification, and review partner. Methodology decisions, empirical design, and finalisation were performed by the human author. The collaboration model is documented in the companion repository's `docs/ARCHITECTURE.md` Parts 5–6.

The author thanks the maintainers of the obra/superpowers skill framework, the contributors of the open agentskills.io standard, and the Anthropic Claude Code team for the `/goal` primitive

whose empirical behavior motivated Iron Laws 11 and 12.

References (APA, 7th edition)

Anthropic. (2026a). *Claude Code* [Software documentation]. https://code.claude.com

Anthropic. (2026b). *Agent Skills best practices* [Software documentation]. https://platform.claude.com/docs/en/ag and-tools/agent-skills/best-practices

Avizienis, A., Laprie, J.-C., Randell, B., & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, *1*(1), 11–33. https://doi.org/10.1109/TDSC.2004.2

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). *Why do multi-agent LLM systems fail?* arXiv. https://arxiv.org/abs/2503.13657

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). *Improving factuality and reasoning in language models through multiagent debate*. ICML 2024. https://arxiv.org/abs/2305.14325

Erdogan, L. E., Lee, N., Kim, S., Moon, S., Furuta, H., Anumanchipalli, G., Keutzer, K., & Gholami, A. (2025). *Plan-and-Act: Improving planning of agents for long-horizon tasks*. ICML 2025. https://arxiv.org/abs/2503.09572

Flores, C. U. (2026b). *Graceful refusals as unsignalled failures: Characterising* `claude -p` *exit-code semantics on agent declines*. See `paper/iron-law-11.md` in the companion repository.

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). *Large language models cannot self-correct reasoning yet*. ICLR 2024. https://arxiv.org/abs/2310.01798

IEEE. (2017). *IEEE standard for system, software, and hardware verification and validation* (IEEE Std 1012-2016). https://doi.org/10.1109/IEEESTD.2017.8055462

International Electrotechnical Commission. (2018). *Failure modes and effects analysis (FMEA and FMECA)* (IEC 60812:2018). https://webstore.iec.ch/en/publication/26359

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). *SWE-bench: Can language models resolve real-world GitHub issues?* ICLR 2024. https://arxiv.org/abs/2310.06770

Kamoi, R., Zhang, Y., Zhang, N., Han, J., & Zhang, R. (2024). When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*. https://arxiv.org/abs/2406.01297

Klein, G. (2007). Performing a project premortem. *Harvard Business Review*, *85*(9), 18–19.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). *Self-Refine: Iterative refinement with self-feedback*. NeurIPS 2023. https://arxiv.org/abs/2303.17651

Meyer, B. (1992). Applying "design by contract." *Computer*, *25*(10), 40–51. https://doi.org/10.1109/2.161279

Mitchell, D. J., Russo, J. E., & Pennington, N. (1989). Back to the future: Temporal perspective in the explanation of events. *Journal of Behavioral Decision Making*, *2*(1), 25–38.

Popper, K. R. (2002). *The logic of scientific discovery*. Routledge. (Original work published 1959)

Shingo, S. (1986). *Zero quality control: Source inspection and the poka-yoke system* (A. P. Dillon, Trans.). Productivity Press. (Original work published 1985)

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). *Reflexion: Language agents with verbal reinforcement learning*. NeurIPS 2023. https://arxiv.org/abs/2303.11366

Tyen, G., Mansoor, H., Cărbune, V., Chen, P., & Mak, T. (2024). LLMs cannot find reasoning errors, but can correct them given the error location. In *Findings of the Association for Computational Linguistics: ACL 2024*. https://arxiv.org/abs/2311.08516

U.S. Department of Defense. (1980). *Procedures for performing a failure mode, effects and criticality analysis* (MIL-STD-1629A). https://everyspec.com/MIL-STD/MIL-STD-1600-1699/MIL_STD_1629A_1556/

Wang, H., Cao, Y., Lin, L., & Chen, J. (2026). *PreFlect: From retrospective to prospective reflection in large language model agents*. arXiv. https://arxiv.org/abs/2602.07187

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*. arXiv. https://arxiv.org/abs/2308.08155

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing reasoning and acting in language models*. ICLR 2023. https://arxiv.org/abs/2210.03629

Yuan, D., Luo, Y., Zhuang, X., Rodrigues, G. R., Zhao, X., Zhang, Y., Jain, P. U., & Stumm, M. (2014). Simple testing can prevent most critical failures: An analysis of production failures in distributed data-intensive systems. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14)* (pp. 249–265). USENIX Association. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/yuan

Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., & Wu, Q. (2025). *Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems*. ICML 2025 Spotlight. https://arxiv.org/abs/2505.00212

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023 D&B. https://arxiv.org/abs/2306.05685

Zhu, K., Liu, Z., Li, B., Tian, M., Yang, Y., Zhang, J., Han, P., Xie, Q., Cui, F., Zhang, W., Ma, X., Yu, X., Ramesh, G., Wu, J., Liu, Z., Lu, P., Zou, J., & You, J. (2025). *Where LLM agents fail and how they can learn from failures*. arXiv. https://arxiv.org/abs/2509.25370

Appendix A — Citation verification status

Every claim in the body cites at most a primary source. The table below records the verification status of each cited reference at the time of preprint submission. The tag scheme is the author's

own and is provided to make verification trivial for a reviewer; tags do not appear in the body for readability.

Tag	Meaning
[V]	Verified directly via the arXiv/journal/HBR URL listed above.
[D]	Anthropic documentation, treated as ground truth for documented tool behaviour.
[E]	Empirical observation from this work or the companion paper.
[O]	Author's operationalisation; not present verbatim in any cited paper.

The references present in version 1.0 were verified by direct URL fetch on 2026-05-17; references added in the July 2026 revision (Flores, 2026b; the classical-lineage entries of Section 2.5) were checked when added, and version 2.0 (2026-11) corrected two venue lines (Tyen et al., 2024; Yao et al., 2023). The MAST 14-mode names are quoted verbatim from Cemri et al. (2025) and listed in the companion skill file `skills/anticipating-shadow-points/mast-checklist.md`. Mitchell et al. (1989) is the primary study of prospective hindsight; the ≈30% figure is Klein's (2007) summary of it, which we could not re-derive from the 1989 paper, and Klein (2007) is also the operational recipe.

The eight references introduced in §2.5 (Avizienis et al., 2004; International Electrotechnical Commission [IEC 60812:2018], 2018; IEEE [Std 1012-2016], 2017; Meyer, 1992; Popper, 1959/2002; Shingo, 1986; U.S. Department of Defense [MIL-STD-1629A], 1980; Yuan et al., 2014) were verified against their primary source — DOI, ISBN, or standard catalogue entry — on 2026-07-08.

Appendix B — Evaluation set and reproducibility

The five engineering evaluations are committed in the companion repository:

- `skills/anticipating-shadow-points/evals/01-supabase-migration.md` (E1)
- `skills/anticipating-shadow-points/evals/02-edge-function-deploy.md` (E2)
- `skills/anticipating-shadow-points/evals/03-refactor-shared-util.md` (E3)
- `skills/anticipating-shadow-points/evals/04-rls-policy-change.md` (E4)
- `skills/anticipating-shadow-points/evals/05-cron-skill-conflict.md` (E5)

Each file contains the INPUT statement, the EXPECTED shadow-points list, and the ACCEPTANCE threshold, all authored by the protocol's designer. The benchmark package in `benchmark/` provides:

- a deterministic 4-layer adjudicator (exact / synonym / Jaccard / unmatched);
- per-run cryptographic provenance (env, code, inputs, outputs and adjudication hashed into a Merkle-style chain);
- a `verify` subcommand that re-walks a run directory and reports any byte-level drift;
- 27 unit tests pinning the determinism contract.

Reproducing every number in Sections 5.3–5.4 (no model calls). From a clone of the repository, `python run_all.py` re-scores the author-constructed fixtures of the mock benchmark run with the shipped adjudicator, recomputes the discrimination matrix, regenerates Figure 2, asserts each number (`tests/test_paper_numbers.py`, which also asserts that the run is labelled `mock`) and verifies a single SHA-256 chain hash over code, tests, frozen inputs, data and results (`output/hash-chain.md`). The author's observations in Section 5.2 cannot be reproduced: their transcripts were not preserved.

Collecting real evidence (costs model calls). Install the benchmark package and run `asp-benchmark run --mode real --model <model>` followed by `asp-benchmark verify <run-id>`. This has not been done for any published number; the protocol to do it properly is in `PREREGISTRATION.md`. The protocol is specified independently of the Python implementation in `benchmark/PROTOCOL.md` so it can be re-implemented in another language.

About the Author

Carlos Ulisses Flores is CTO and Chief Researcher at Codex Hash Research Laboratory, an independent R&D lab in São Paulo, Brazil. He builds production systems where silent failure is expensive — historically in algorithmic trading, embedded IoT and edge cryptography, and currently in agentic-AI tooling. The ASP skill described in this whitepaper grew out of that latter line of work; the empirical finding reported in the companion whitepaper grew out of using Anthropic's `claude -p` runner as an autonomous execution kernel and watching it return exit code 0 on a graceful agent refusal.

He is an MSc candidate in Artificial Intelligence at American Global Tech University and writes in Portuguese, English, Spanish, and Italian.

Identifiers · ORCID: 0000-0002-6034-7765 · Lattes: 6905246706890561 **Contact** · c.ulisses@gmail.com · https://ulissesflores.com

---

© 2026 Carlos Ulisses Flores · Codex Hash Research Laboratory · São Paulo, Brazil Paper text licensed under CC BY 4.0 · Companion software licensed under MIT

End of whitepaper. References: 28 entries (26 primary sources and 2 Anthropic documentation pages, spanning 1980–2026). Word count: approximately 5,500 words excluding tables, code blocks, and appendices.